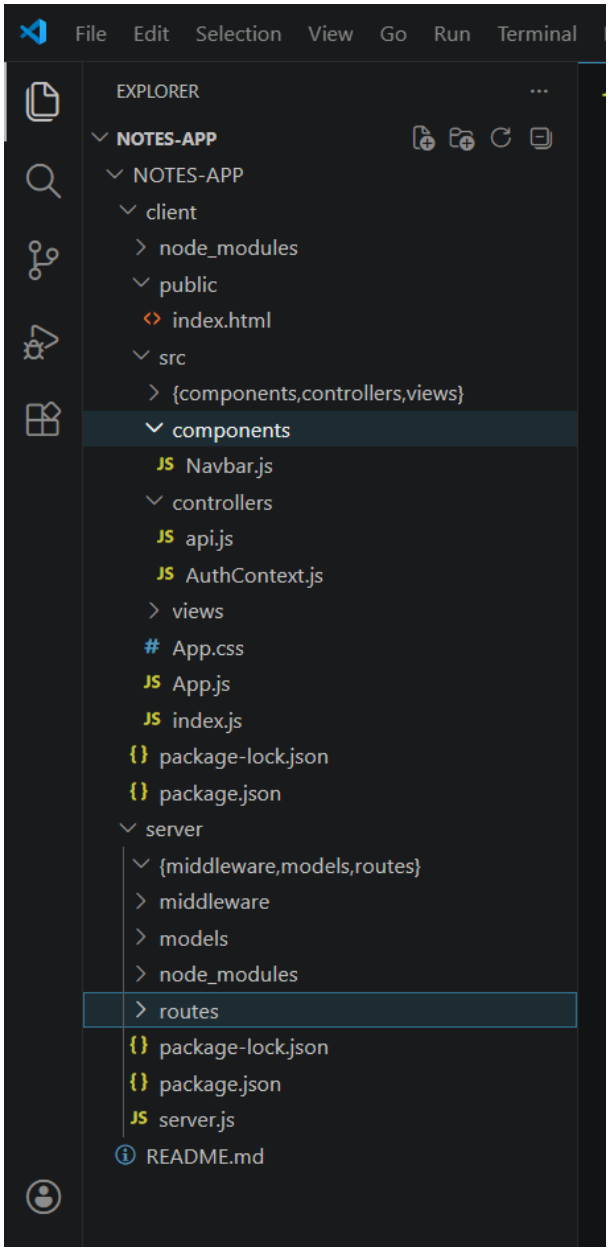
MERN STACK APPLICATION-INTERNSHIP TRACKER SYSTEM

AIM:

To design and develop a full-stack web application using the **MERN Stack (MongoDB, Express.js, React.js, Node.js)** that enables students to efficiently track, manage, and organize their internship applications, including details such as application status, company information, deadlines, and progress updates in a centralized platform.

SOURCE CODE:

FILE STRUCTURE:



Index.html:

```html
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Smart Internship & Placement System</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&fam
ily=Space+Grotesk:wght@400;500;600;700&display=swap" rel="stylesheet">
</head>
<body>
<noscript>You need to enable JavaScript to run this app.</noscript>
<div id="root"></div>
</body>
</html>
```

Components:

Navbar.js:

```javascript
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../controllers/AuthContext';

const Navbar = () => {
const { user, logout } = useAuth();
const navigate = useNavigate();

const handleLogout = () => { logout(); navigate('/login'); };

const links = {
student: [
{ to: '/student/dashboard', label: '🏠 Dashboard' },
{ to: '/internships', label: '🔍 Internships' },
{ to: '/student/applications', label: '📋 My Applications' },
{ to: '/student/profile', label: '👤 Profile' },
],
company: [
{ to: '/company/dashboard', label: '🏠 Dashboard' },
{ to: '/company/post', label: '➕ Post Internship' },
{ to: '/company/profile', label: '🏢 Company Profile' },
```

```
],
admin: [
{ to: '/admin/dashboard', label: '📊 Dashboard' },
{ to: '/internships', label: '📌 Internships' },
],
};

return (
<nav className="navbar">
<Link to="/" className="navbar-brand">🎓 PlacementPro</Link>
{user && (
<div className="navbar-links">
          {(links[user.role] || []).map(l => <Link key={l.to}
to={l.to}>{l.label}</Link>)}
<span className={`badge-role badge-${user.role}`}>{user.role}</span>
<span style={{ fontSize: '0.85rem', color: '#64748b', fontWeight:
500 }}>{user.name}</span>
<button className="btn-logout" onClick={handleLogout}>Logout</button>
</div>
      )}
    </nav>
  );
}
;

export default Navbar;
```

Controllers:

Api.js:

```
import axios from 'axios';

const api = axios.create({ baseURL: '/api' });

api.interceptors.request.use((config) => {
const token = localStorage.getItem('token');
if (token) config.headers.Authorization = `Bearer ${token}`;
return config;
});
```

```
api.interceptors.response.use(
  res => res,
  err => {
  if (err.response?.status === 401) {
  localStorage.removeItem('token');
  localStorage.removeItem('user'); window.location.href
  = '/login';
  }
  return Promise.reject(err);
  }
  );

  export default api
```

Authcontext.js:

```
  import React, { createContext, useContext, useState, useEffect } from 'react';
  import api from '../controllers/api';

  const AuthContext = createContext();
  export const useAuth = () => useContext(AuthContext);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(() => {
      try { return JSON.parse(localStorage.getItem('user')); } catch { return
  null; }
  });
  const [loading, setLoading] = useState(false);

  const login = async (email, password) => {
  const { data } = await api.post('/auth/login', { email, password });
  localStorage.setItem('token', data.token); localStorage.setItem('user',
  JSON.stringify(data.user)); setUser(data.user);
  return data.user;
  };

  const register = async (payload) => {
  const { data } = await api.post('/auth/register', payload);
  localStorage.setItem('token', data.token); localStorage.setItem('user',
  JSON.stringify(data.user)); setUser(data.user);
  return data.user;
  };
```

```
const logout = () => { localStorage.removeItem('token');
  localStorage.removeItem('user');
  setUser(null);
  };

  return (
  <AuthContext.Provider value={{ user, login, register, logout, loading }}>
  {children}
  </AuthContext.Provider>
  );
  };
```

Server:

Middleware:

Auth.js:

```
    const jwt = require('jsonwebtoken');
    const User = require('../models/User');

  const protect = async (req, res, next) => {
    let token;
      if (req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')) {
    try {
    token = req.headers.authorization.split(' ')[1];
          const decoded = jwt.verify(token, process.env.JWT_SECRET ||
    'smartplacement_secret_2024');
    req.user = await User.findById(decoded.id).select('-password'); next();
    } catch (err) {
    return res.status(401).json({ message: 'Token invalid or expired' });
    }
    } else {
    return res.status(401).json({ message: 'No token provided' });
    }
    };

  const authorize = (...roles) => {
    return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: `Access denied for role:
    ${req.user.role}` });
    }
    next();
    };
    };

    module.exports = { protect, authorize
```

Models:

Application.js:

```
const mongoose = require('mongoose');

const applicationSchema = new mongoose.Schema({
  internship: { type: mongoose.Schema.Types.ObjectId, ref: 'Internship',
required: true },
student: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
status: {
type: String,
    enum: ['pending', 'reviewed', 'shortlisted', 'interview_scheduled',
'selected', 'rejected', 'placed'],
default: 'pending'
},
coverLetter: { type: String },
interview: {
scheduledAt: { type: Date },
mode: { type: String, enum: ['online', 'offline', 'phone'] }, notes:
{ type: String },
meetLink: { type: String }
},
feedback: {
rating: { type: Number, min: 1, max: 5 },
comment: { type: String },
givenAt: { type: Date }
},
convertedToPlacement: { type: Boolean, default: false },
appliedAt: { type: Date, default: Date.now },
updatedAt: { type: Date, default: Date.now }
});

// Prevent duplicate applications
applicationSchema.index({ internship: 1, student: 1 }, { unique: true });

module.exports = mongoose.model('Application', applicationSchema);
```

Companyprofile.js:

```
const mongoose = require('mongoose');
const companyProfileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true,
unique: true },


companyName: { type: String, required: true },
industry: { type: String },
website: { type: String },
description: { type: String },
location: { type: String },
logoUrl: { type: String },
contactEmail: { type: String },
contactPhone: { type: String },
verified: { type: Boolean, default: false },
updatedAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('CompanyProfile', companyProfileSchema);
```

Internship.js:

```
const mongoose = require('mongoose');

const internshipSchema = new mongoose.Schema({
company: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
title: { type: String, required: true },
description: { type: String, required: true },
location: { type: String },
  type: { type: String, enum: ['remote', 'onsite', 'hybrid'], default:
'onsite' },
duration: { type: String },
stipend: { type: Number, default: 0 },
openings: { type: Number, default: 1 },
deadline: { type: Date, required: true },
eligibility: {
minCGPA: { type: Number, default: 0 },
requiredSkills: [{ type: String }], requirePassport:
{ type: Boolean, default: false },
requiredCertifications: [{ type: String }],
departments: [{ type: String }]
},
status: { type: String, enum: ['open', 'closed', 'filled'], default: 'open' },
canConvertToPlacement: { type: Boolean, default: false },
createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Internship', internshipSchema);
```

Studentprofile.js:

```
const mongoose = require('mongoose');

const studentProfileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true,
unique: true },
  rollNumber: { type: String },
  department: { type: String },
  year: { type: Number },
  cgpa: { type: Number, default: 0 },
  skills: [{ type: String }],
  certifications: [{ type: String }],
  hasPassport: { type: Boolean, default: false },
  resumeUrl: { type: String },
  phone: { type: String },
  bio: { type: String },
  linkedIn: { type: String },
  github: { type: String },
  placedAt: { type: String, default: null },
  isPlaced: { type: Boolean, default: false },
  updatedAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('StudentProfile', studentProfileSchema);
```

User.js:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['student', 'company', 'admin'], default:
'student' },
  createdAt: { type: Date, default: Date.now }
```

```
});

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.matchPassword = async function (entered) {
  return await bcrypt.compare(entered, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Package.json:

```
{
  "name": "smart-placement-server",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.0.3",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

Server.js:

```
const express = require(‘express’);
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');

dotenv.config();

const app = express();

app.use(cors({ origin: process.env.CLIENT_URL || 'http://localhost:3000',
credentials: true }));
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/internships', require('./routes/internship'));
app.use('/api/applications', require('./routes/application'));
app.use('/api/analytics', require('./routes/analytics'));
app.use('/api/users', require('./routes/users'));

const MONGO_URI = process.env.MONGO_URI ||
'mongodb+srv://tamilyazh2023_db_user:luKQaaoXJX6i4ihV@intertracker.6rpno0f.mongodb.net/?appName=InterTracker';

mongoose.connect(MONGO_URI)
  .then(() => console.log('✅ MongoDB Connected'))
  .catch(err => console.error('❌ MongoDB Error:', err));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`)
```